Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №34

ТЕМА: Створення програм з використанням користувацьких функцій.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- закріпити навички створення й виклику власних функцій;
- навчитися розбивати складну задачу на кілька простіших функцій;
- ознайомитися з поняттям локальних і глобальних змінних;
- розвинути навички побудови структурованих програм із функціями.

Теоретичні відомості

1. Декомпозиція задачі на функції

Декомпозиція — це розбиття складної задачі на кілька простіших, кожна з яких оформлюється як окрема функція. Такий підхід робить програму зрозумілішою, спрощує пошук помилок (кожну функцію можна перевірити окремо) і дозволяє повторно використовувати вже написані функції в інших задачах.

2. Локальні та глобальні змінні

Змінна, оголошена всередині функції, називається локальною — вона існує тільки під час виконання цієї функції і недоступна ззовні. Змінна, оголошена поза будь-якою функцією (на «верхньому рівні» програми), називається глобальною.

```
def calculate():
    local_value = 100   # локальна змінна
    print(local_value)

calculate()
print(local_value)   # помилка: local_value тут не існує
```

Функція може читати значення глобальної змінної, але щоб змінити її значення всередині функції, потрібне спеціальне ключове слово global — утім, у навчальних програмах цього краще уникати і натомість передавати потрібні дані через параметри та повертати результат через return, що є набагато акуратнішим стилем програмування.

3. Приклад: програма з кількома функціями

Розглянемо програму-калькулятор площ фігур, у якій кожна фігура обчислюється окремою функцією:

```
import math

def area_rectangle(width, height):
    return width * height

def area_circle(radius):
    return math.pi * radius ** 2

def area_triangle(base, height):
    return 0.5 * base * height

print(f'Площа прямокутника: {area_rectangle(4, 5)}')
print(f'Площа кола: {area_circle(3):.2f}')
print(f'Площа трикутника: {area_triangle(6, 4)}')
```

Результат: *Площа прямокутника: 20*
Площа кола: 28.27
Площа трикутника: 12.0

Головна частина програми (виклики print) стала короткою й зрозумілою саме завдяки тому, що обчислення винесено в окремі, промовисто названі функції.

4. Функція, що викликає іншу функцію

Функції можуть викликати одна одну — це дозволяє будувати складніші обчислення з простіших «будівельних блоків»:

```
def is_even(n):
    return n % 2 == 0

def count_evens(numbers):
    count = 0
    for num in numbers:
        if is_even(num):
            count = count + 1
    return count

print(count_evens([1, 2, 3, 4, 5, 6]))
```

Результат: *3*

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані завдань 1–5 залежать від N:

Завдання	**Дані варіанта**	**Приклад N = 7**
1	набір (N, N + 3)	(7, 10)
2	список [N, N + 4, 2N, N + 6, 1, 2, 9]	[7, 11, 14, 13, 1, 2, 9]
3	температура N + 10 °C	17 °C
4	число 1000·N + 234	7234
5	список [N, 3, 12, N + 5, 8]	[7, 3, 12, 12, 8]

б) Кожне завдання — окремий файл Прізвище_ПР34_зN.py (наприклад, Ivanenko_PR34_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Кожну дію виносьте в окрему функцію; функції повертають значення (return), а виводить їх основна частина програми. Імена функцій — точно як в умові. Заборонено sum() і max() у завданні 5 та перетворення числа на рядок у завданні 4.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР34_з1.py. Напишіть окремі функції area(a, b) і perimeter(a, b) для прямокутника й викличте їх для двох наборів даних: (3, 4) і (N, N + 3).

   Приклад (N = 7): (3, 4) → площа 12, периметр 14; (7, 10) → площа 70, периметр 34.

2. Напишіть функцію is_prime(n) (True/False; числа менші за 2 не прості) і функцію count_primes(numbers), яка використовує is_prime() для підрахунку простих чисел списку вашого варіанта.

   Приклад (N = 7): [7, 11, 14, 13, 1, 2, 9] → 4 простих (7, 11, 13, 2); is_prime(1) → False.

3. Напишіть функції celsius_to_fahrenheit(c) і fahrenheit_to_celsius(f) та перевірте на температурі вашого варіанта (N + 10 °C), що застосування

обох функцій підряд повертає початкове значення: виведіть «Перевірка пройдена» або «Помилка».

Приклад (N = 7): 17°C → 62.6°F → 17.0°C, «Перевірка пройдена».

4. Напишіть функцію sum_of_digits(n), що повертає суму цифр цілого числа лише через // та % (без рядків); від'ємне число обробіть через abs(). Перевірте на числі 1000·N + 234.

   Приклад (N = 7): 7234 → 16; 12345 → 15; -482 → 14; 0 → 0.

5. Напишіть функції list_sum(numbers), list_average(numbers), list_max(numbers) (без sum() і max()) і викличте їх з основної частини для одного списку вашого варіанта: «Сума: …», «Середнє: …» (2 знаки), «Максимум: …».

   Приклад (N = 7): [7, 3, 12, 12, 8] → «Сума: 42», «Середнє: 8.40», «Максимум: 12».

6. Завдання підвищеної складності*. «Текстове меню»: користувач вводить номер пункту 1–4 (1 — площа прямокутника, 2 — температура, 3 — просте число, 4 — сума цифр) і необхідні дані; програма викликає відповідну функцію через if-elif-else. Для іншого номера виведіть «Невідомий пункт меню».

   Приклад: 3, потім 17 → «Просте»; 4, потім 482 → «Сума цифр: 14»; 9 → «Невідомий пункт меню».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, функція виводить замість return, числа менші за 2 визнано простими, використано str() у завданні 4, пропущено пункт меню) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, додати нову функцію чи змінити умову перевірки). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Що таке декомпозиція задачі і чому вона корисна при написанні програм?
2. Чим локальна змінна відрізняється від глобальної?
3. Чи можна звернутися до локальної змінної функції поза цією функцією? Чому?
4. Наведіть приклад ситуації, коли одна функція викликає іншу.
5. Які переваги дає розбиття калькулятора площ на окремі функції для кожної фігури замість одного великого блоку коду?